FINAL PROJECT REPORT

SYNCLAB

Centralized Workshop & Laboratory Allocation Platform

Group 01

NANO2142 - Introduction to Software Development

Department of Nano Science Technology
Faculty of Technology
Wayamba University of Sri Lanka

Report field	Details
Submitted to	Dr. U.S. Liyanarachchi
Document status	Final submission documentation based on the active program
Prepared date	07 October 2026
Implementation	C++17 / Qt 6 / SQLite
Evidence	Compiled integration tests and executed SQL outputs

Preparation date is not a confirmed submission deadline. Group-member attribution requires group confirmation. All example records and screenshots use fictional demonstration data.

REPORT / 02

Executive overview and contents

SYNCLAB supports allocating university workshops and laboratories through a guided desktop workflow. Students choose a facility, date and available time slots, then provide identity, supervisor and project details. The system creates a Pending request and displays an allocation key. Students use the key to revise or remove the request. Administrators review workshop-scoped records, adjust status and remarks, filter by date and prepare correspondence.

The implementation uses Qt Widgets for the interface and SQLite for local persistence. Four normalized tables represent workshops, students, allocation records and slot memberships. A database interface separates persistence operations from the window, while AllocationRules centralizes form checks and status preparation. Transactions prevent partial saves and check active occupancy before accepting a change.

The active project compiled with Qt 6.11.2 and MinGW 13.1.0. The compiled harness passed 10 test groups containing 69 assertions. Twenty independent SQL checks and nineteen additional UI update/delete assertions passed, and five sample queries have complete exported outputs. These results establish tested behavior with isolated data; they do not establish automatic email delivery or deployment on every computer.

Report contents

Page	Content
03	Introduction, background, problem and objectives
04	Functional and non-functional requirements
05	Use cases and architecture
06	UI/UX design
07	ER model, relational schema and normalization
08	Class relationships and OOP
09	Implementation decisions
10	Testing strategy and results
11	Limitations, improvements and group contributions
12	Conclusion and references

REPORT / 03

Background, problem and objectives

Academic setting

Workshops and laboratories are shared facilities used for practical exercises and project work. A useful allocation tool must make facility choices, dates and reservation status visible to students and the staff who review access. This project applies the course requirements for a graphical interface, an SQL database and C++ object-oriented design to that academic setting. It presents a desktop prototype suitable for a local assessment demonstration.

Problem definition

The scheduling problem is to retain student and project details alongside the requested laboratory time, prevent overlapping active allocations and allow staff to review the request. A schedule also needs to remain understandable after a request is approved, rejected, changed or deleted. Repeating identity details and multiple selected time periods inside a single unstructured record makes consistent updates and availability checking harder.

SYNCLAB addresses these needs with an ordered booking flow and a structured relational model. The student supplies the required details once per booking interaction. The database stores a shared profile by student index, while each allocation holds its own date, project and supervisor facts. Each selected period occupies a separate child row. Pending and approved requests reserve their slots, and rejected requests release them.

Project objectives

- Provide clear selection, date/slot and data-entry screens for ten laboratory/workshop categories.
- Persist requests and reconstruct their details through normalized SQLite joins.
- Validate required inputs and prevent active conflicts using transaction-protected checks.
- Support student key-based editing and workshop administrator review, filtering, status, remarks and deletion.
- Demonstrate C++ encapsulation, inheritance, virtual-method dispatch and Qt event handling.

Context basis: Group Software Development Project Information Sheet and the active source. No survey findings or measured institutional savings were supplied or claimed.

REPORT / 04

System requirements

Functional requirements

ID	Required behavior	Implementation basis
FR-01	Choose a workshop from ten categories	Catalog and selection cards
FR-02	Choose a current/future date and free slot(s)	Calendar and allocation validation
FR-03	Save student, supervisor and project information	Form and StudentRecord
FR-04	Return a key for student edits/deletion	New key and edit flow
FR-05	Review status and save remarks	Administrator table
FR-06	Filter by workshop/date	SQLite queries and admin calendar
FR-07	Maintain passcodes and appearance	System settings / QSettings
FR-08	Prepare pending/rejection email draft	mailto integration

Non-functional requirements

Usability requires an ordered workflow, clear primary actions and specific feedback when an input is invalid. Reliability requires transaction rollback and foreign-key integrity so related facts remain consistent after a failed save or deletion. Maintainability requires an explicit active build manifest and separate domain, rules, database and utility responsibilities.

The Windows bundle includes Qt and QSQLITE; source builds need a matching Qt 6 Desktop kit. Storage must be writable. Keys/passcodes provide lightweight prototype access. The packaging manifest sets SOURCE_DIR to a dot and its launcher sets the working directory to Windows_x64, selecting its bundled data folder.

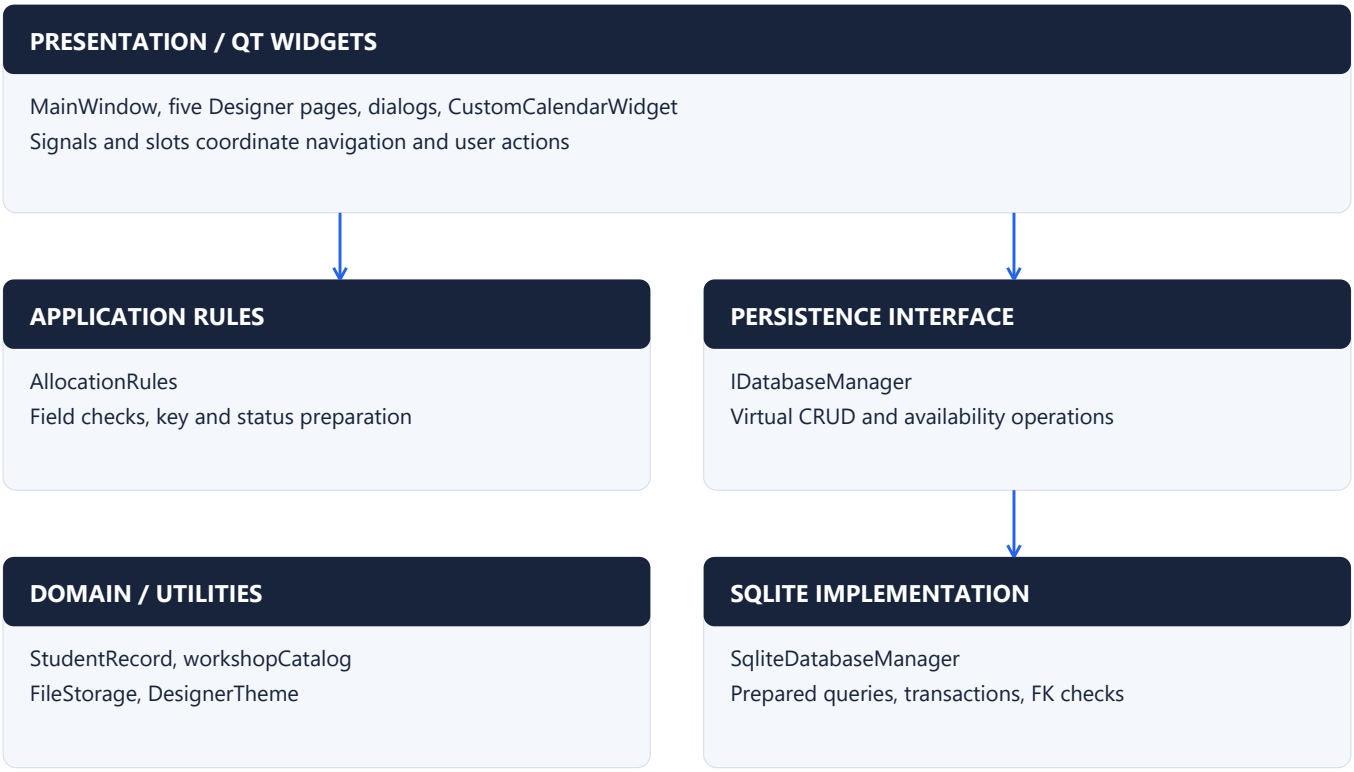
Acceptance boundary

Required create/read/update/delete operations, active slot protection and meaningful query outputs are covered by executed tests. Broader accessibility, stress-load, delivery tracking and remote simultaneous users remain future evaluation topics rather than verified guarantees.

REPORT / 05

Use cases and high-level architecture

Use case	Primary responsibility
UC-01 Create a workshop allocation	Student
UC-02 Find and change an existing allocation	Student
UC-03 Review and manage workshop requests	Workshop administrator
UC-04 Inspect allocations by date	Student / administrator
UC-05 Configure access and compose correspondence	System administrator / workshop administrator



MainWindow coordinates screens and application actions. AllocationRules validates and prepares record state. IDatabaseManager defines the CRUD, query and availability contract. SqliteDatabaseManager implements it with SQLite. StudentRecord carries values between these layers; catalog and storage helpers supply shared metadata and paths.

Detailed use cases: section A1. Architecture reflects compiled sources rather than excluded older file splits.

REPORT / 06

UI/UX design

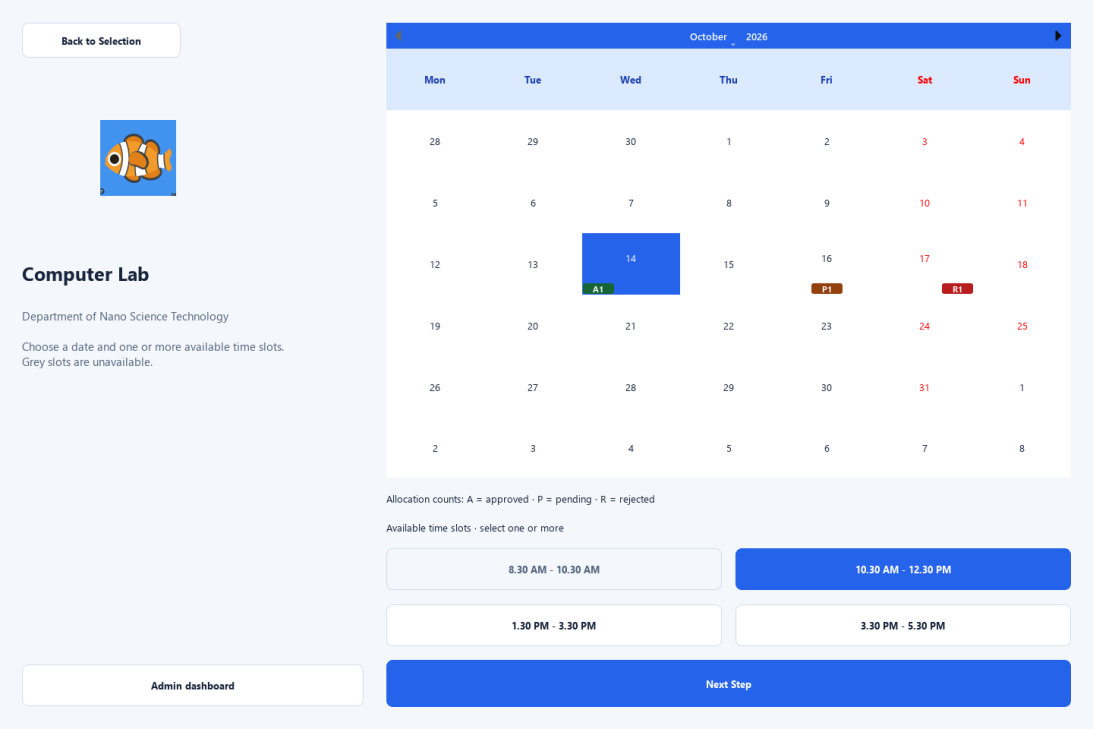


Figure 1. Actual compiled calendar screen with fictional test data.

Guided student interaction

Welcome provides the initial access choices. The workshop page establishes category context. The calendar shows available daily slots and date-level status counts. The final form collects name, index, department, email, supervisor and project, with an optional description. Continuation controls remain disabled when a category or slot selection is missing.

Administrator interaction

The dashboard opens for the current workshop after passcode verification. A date calendar filters records, and Show all restores the complete category list. Selecting a row enables edit/delete and email actions. A status dropdown and remark editor write changes, refresh calendar information and show failures if the database refuses a request.

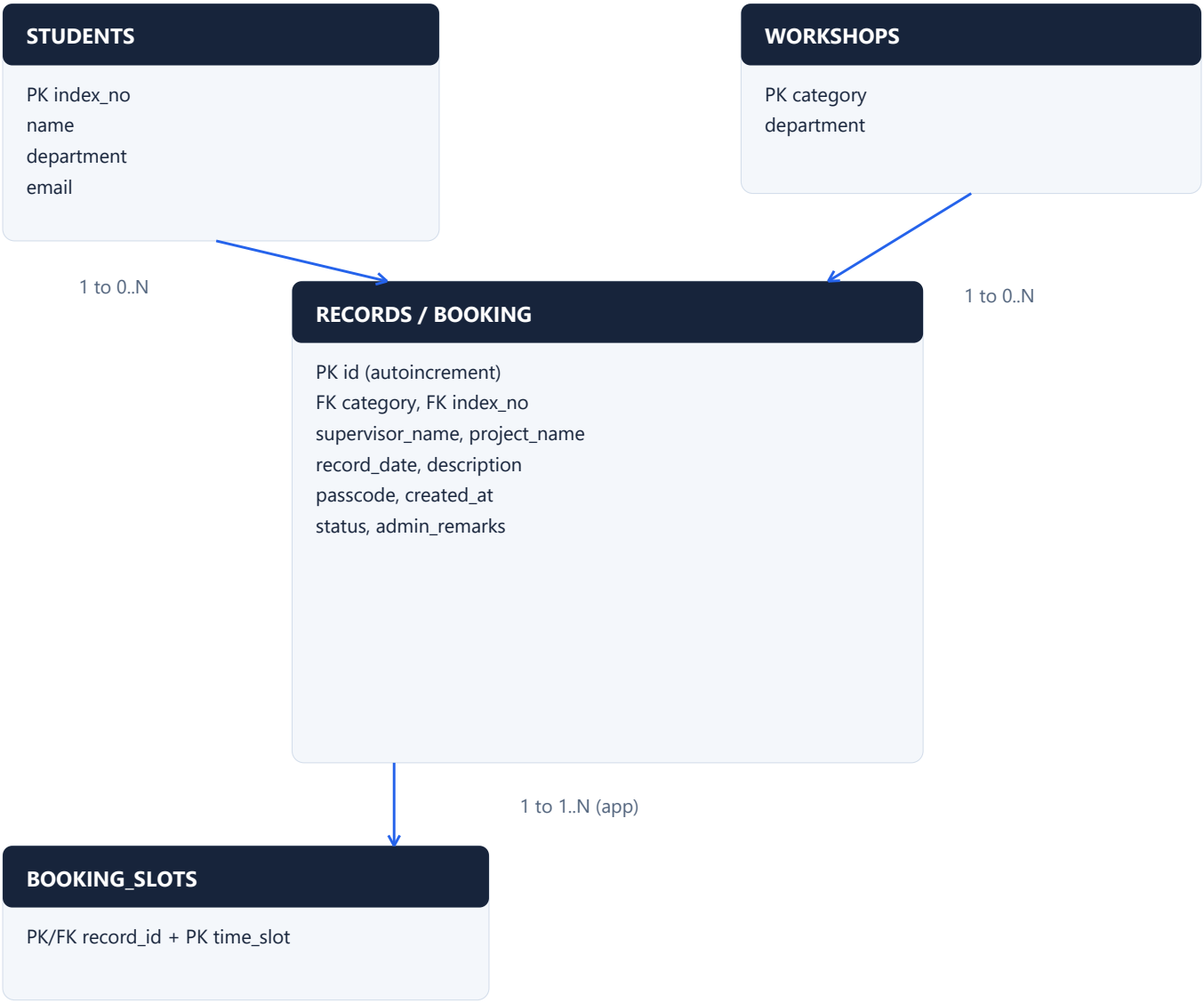
Visual and feedback decisions

The source uses Segoe UI, Qt Fusion and shared blue/gray styling, with dark-theme and wallpaper settings. The calendar paints status counts by booking records. Form validation focuses the first failing control. Deletion and discard prompts reduce accidental loss. Actual screenshot pages and simplified wireframes accompany the design document.

Testing captures establish selected implemented states. They do not establish usability for every screen size or an accessibility conformance result.

REPORT / 07

Database design and normalization



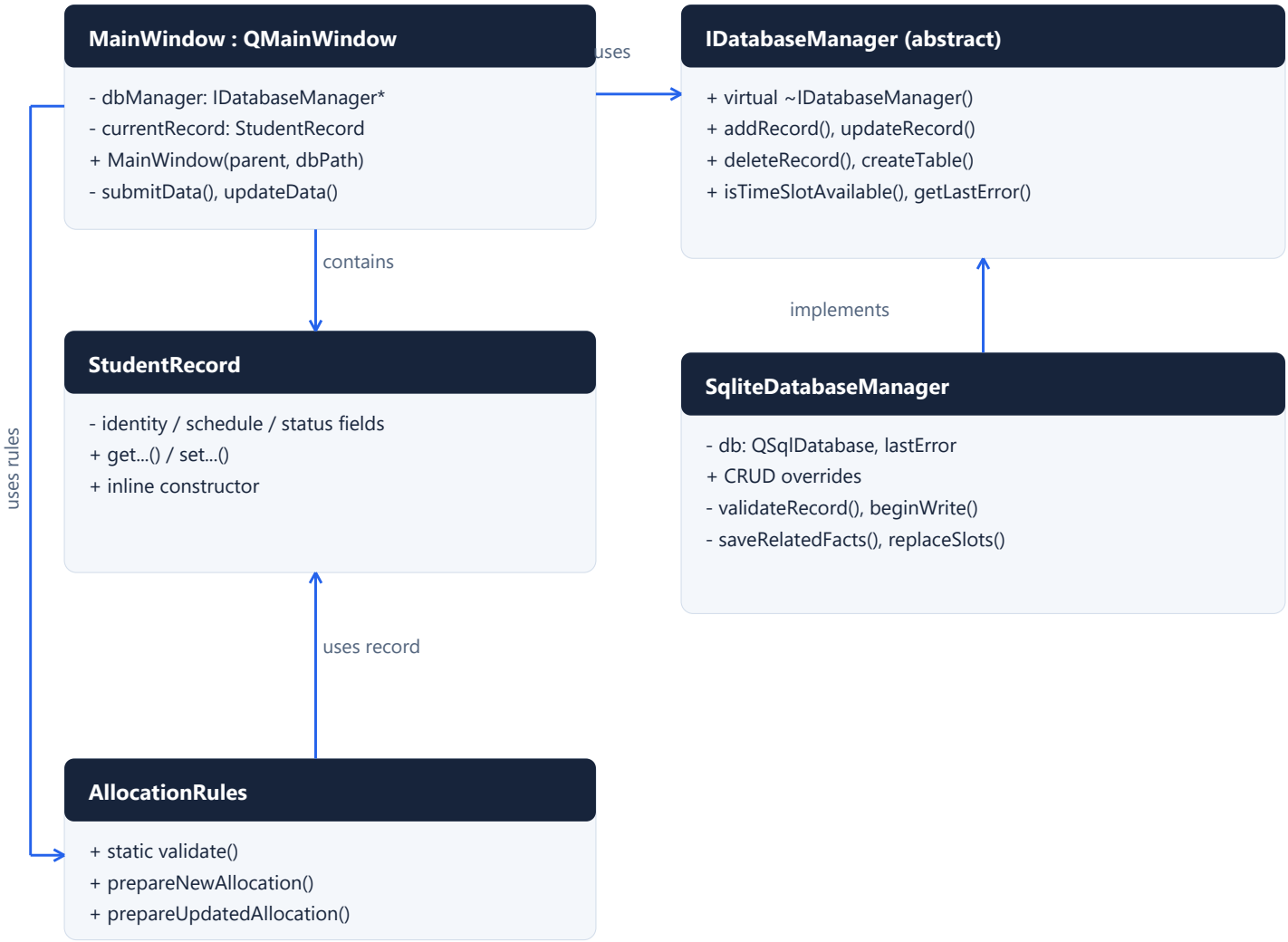
workshops(category PK, department) and students(index_no PK, name, department, email) store independent reference/profile facts. records uses an auto-increment id and foreign keys to both parents, alongside supervisor, project, date, description, key, creation time, status and remarks. booking_slots(record_id, time_slot) uses a composite primary key and cascades when its record is deleted.

One slot per child row satisfies atomic storage. Separating profiles and workshop departments removes repetition and transitive dependence. Under the stated key dependencies and no additional independent multivalued facts, the design is compatible with BCNF/4NF. The SQL schema enforces keys and status/slot checks. Application transactions enforce cross-record slot and key rules.

Detailed DDL, constraints, dictionaries and dependency reasoning are in section A3. SQL alone permits a zero-slot parent and duplicate allocation keys across records.

REPORT / 08

Software classes and OOP



Object-oriented decisions

StudentRecord encapsulates private values behind accessors. The database manager hides its connection and transaction helpers. IDatabaseManager provides an abstract persistence boundary with virtual operations and a virtual destructor. MainWindow invokes this interface, and SqliteDatabaseManager overrides its behavior.

Qt inheritance supplies the main window, dialogs and custom calendar lifecycle. Runtime polymorphism also invokes the calendar paintCell override for status decoration. MainWindow contains its current record and owns the database manager. Signals and slots provide event-driven communication from buttons, calendar changes and settings signals.

A separate five-page OOP artifact contains the two-page design note and creation/status sequence diagrams. The architecture does not claim a full dependency-injection framework or a separate service layer.

REPORT / 09

Implementation details and decisions

Build and organization

StudentWorkshopApp.pro specifies C++17, Qt Core/GUI/Widgets/SQL, generated Designer forms and the resource collection. It rejects Qt versions below 6. The active entry point sets the application identity, font and Fusion style, then opens MainWindow. Meaningful source folders separate UI, database, models, logic, resources and utilities.

Older split files remain outside SOURCES; active UI/database units are monolithic and StudentRecord methods are inline. The 73-file B5 snapshot matches the original. B6 uses a separate reproducible .pro changing only deployment paths and SOURCE_DIR to a dot. Application C++ is unchanged.

Data consistency

The SQLite implementation uses prepared bindings and BEGIN IMMEDIATE for writes. Related student/workshop facts and slot replacement commit together. Availability checks ignore Reject records and exclude the record being edited. Reapproval checks its selected slots again, preventing an active conflict after a slot was reused. Updating a student profile affects joined displays of that student index across bookings.

State and recovery

New allocations begin Pending and receive a zero-padded four-digit key. A collision returns an error for resubmission. Student updates reset status to Pending; administrator updates preserve it. createTable migrates older data with a separate disk backup, staging tables, version checking and foreign-key verification. Settings reside outside SQLite in QSettings.

External behavior

Pending and rejection email actions save the requested status and open a mailto draft. An external email application handles review and sending. The application cannot confirm delivery. The current build has no attachment upload, export/report button or routine in-app recovery interface.

Implementation evidence: active main.cpp, .pro, mainwindow.cpp, AllocationRules, SqliteDatabaseManager and FileStorage.

REPORT / 10

Testing strategy and results

Verification separates source inspection from executed behavior. A production release build checks the active manifest and compiler integration. A compiled harness uses isolated database/settings storage, calls actual persistence and rule methods, exercises selected UI handlers and renders widgets. A separate sqlite3 run executes the exact submitted DDL and sample INSERT data. A later UI continuation updates and deletes a copied demonstration record, passing nineteen further assertions with before/after screenshots.

Verification	Actual result
Production release build	Success with Qt 6.11.2 / MinGW 13.1.0
Integration test groups	10 / 10 passed
Integration assertions	69 executed, 0 failed
Independent SQL checks	20 / 20 passed
Query exports	5 SELECT scripts with complete CSV/JSON tables

Behavior covered

Assertions cover normalized schema creation, booking creation and filtering, update/delete behavior, child slot cascade, rejected-slot release, occupied-slot protection, key collisions and relevant rule validation. Captured UI states show the main student flow and populated admin view. Handled invalid input and wrong passcodes are recorded as expected outcomes rather than successful operations.

Assessment of the evidence

The exported queries reconstruct joined booking details, count workshop use including unused categories, find free slots on a chosen date, summarize status workload and list one student history. The independent SQL checks include key/constraint failures, foreign-key integrity and deletion cascade. Reproduction files and detailed expected/observed values are retained.

Remaining acceptance work

The delivered production bundle passed an isolated startup smoke check with Qt/MinGW modules loaded from its own folder. The live assessment still needs target-display, data-path, email-handler and group-participation checks. Full legacy migration, installer, accessibility and stress coverage is not established.

Evidence artifacts: section C and section D, including runtime_results.json and sql_verification_results.json.

REPORT / 11

Limitations, improvements and contributions

Current limitations

• Data is local SQLite. There is no web/mobile client, remote synchronization or server-based shared workflow.

• Four-digit allocation keys and locally stored passcodes provide limited protection. The prototype has no account lockout, password hashing or access audit.

• The catalog and four time periods are fixed in source. Equipment capacity, closures and variable durations are not managed dynamically.

• The email workflow prepares drafts and requires manual sending. Attachments, built-in report export and routine in-app backup/restore are absent.

• Student profile facts are current shared values. Historical displays do not retain a separate immutable snapshot of name/department/email.

Possible future improvements

Priorities include stronger identity/access, a user-configurable data path, structured backup/restore and a staff audit trail. A later architecture could separate window orchestration into controllers and inject the persistence interface for broader testing. Capacity and closure rules, editable catalog data and report export should be introduced with matching database constraints and acceptance tests.

Individual contributions

The supplied project documents identify Group 01 but do not provide a verified member roster or individual contribution allocation. The group must add and confirm its member names, index numbers and actual contributions before final submission. No individual attribution has been invented.

Recommended final declaration should identify each member, index number, actual implementation/documentation/testing contribution and demo role. Proposed speaking roles in the demo plan are not proof of past work.

REPORT / 12

Conclusion and references

Conclusion

SYNCLAB integrates a guided Qt interface, normalized local relational data and C++ object-oriented components for workshop and laboratory allocations. Students can create and revise requests, while administrators manage status, remarks and workshop/date-scoped views. Atomic slot memberships and transaction-protected checks keep schedule information coherent when a request changes or is rejected.

The successful production build, 10 passing integration groups and twenty independent SQL checks provide reproducible evidence for the tested implementation. Actual screens and complete query exports support assessment. The prototype remains bounded by local storage, lightweight access control and external email handling. Future work should extend those boundaries with explicit requirements and new tests.

Source references

Ref.	Source used
S01	Final Submission Package Guide.pdf and Final Submission Checklist.pdf - structure, 19 checks and page expectations.
S02	Group Software Development Project - Information Sheet.pdf - NANO2142, purpose, deliverables and assessment context.
S03	StudentWorkshopApp.pro and main.cpp - active manifest, language/modules and startup.
S04	ui/mainwindow.h / ui/mainwindow.cpp and active Designer forms - screens, workflow and access.
S05	database/idatabasemanager.h, databasemanager.h / .cpp - schema, migration and persistence.
S06	models/studentrecord.h, logic/allocationrules.h / .cpp - encapsulation, validation and state.
S07	utilities/filestorage.cpp / workshopcatalog.cpp and customcalendarwidget - paths/catalog/calendar.
S08	runtime_results.json and SQL verification/query exports - executed results with fictional data.

Earlier database query sheets informed project naming but contain outdated schema proposals. The active source is authoritative for implementation claims. No external literature, measured usage data or additional member attribution was invented.